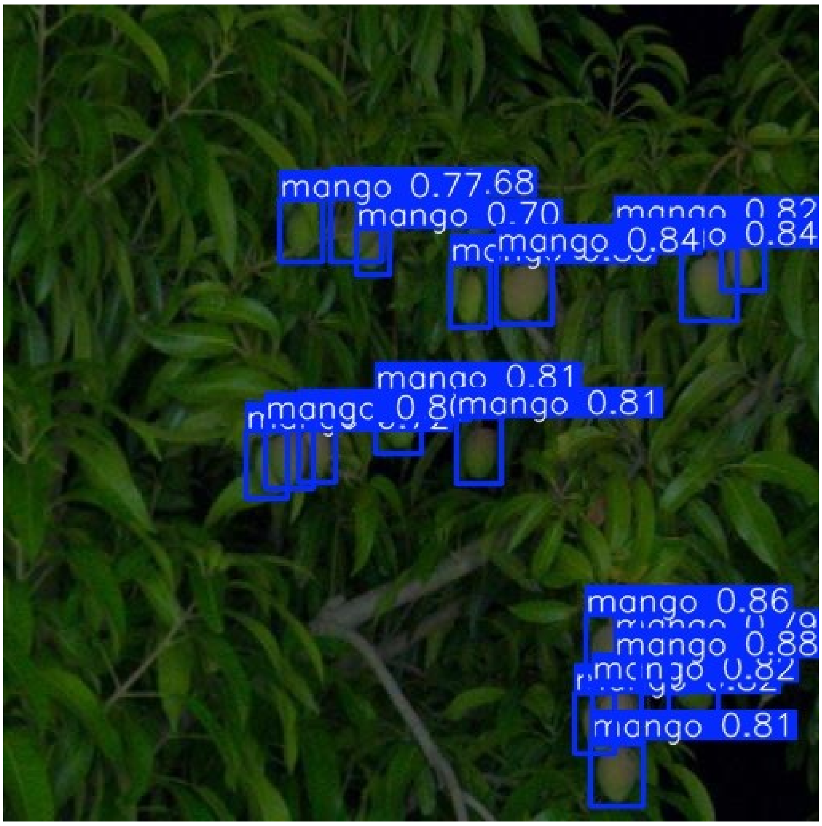
# YOLOv11-n Mango Detection in Orchard Images

## Training, Evaluation and Comparison with DETR ResNet-50




*Example YOLOv11-n detection output on a held-out orchard image.*

**Jerald Christopher Bucud**

Computer Vision Portfolio Project | July 2026

**Portfolio publication note**

This report documents an independent portfolio project. It is not an IEEE Access publication and does not represent endorsement by IEEE or any university.

## Executive Summary

This project evaluated YOLOv11-n and DETR ResNet-50 for single-class mango detection in orchard images. Both models were trained and tested using the same annotated dataset containing 1,730 images: 1,384 for training, 260 for validation and 86 for final testing. The objective was to identify a detector that combined accurate mango localisation with practical inference efficiency.

YOLOv11-n achieved the strongest overall held-out test performance, with mAP@0.5 of 0.989359, mAP@0.5:0.95 of 0.704345, precision of 0.936436 and recall of 0.971182. DETR ResNet-50 achieved mAP@0.5 of 0.960000, mAP@0.5:0.95 of 0.543000, precision of 0.801909 and recall of 0.968300. Under the recorded CUDA benchmark, YOLOv11-n averaged 13.302 ms per image, whereas DETR ResNet-50 averaged 78.436 ms per image.

**Key finding**

YOLOv11-n was selected as the final portfolio model because it produced stronger localisation, fewer false-positive detections and substantially faster inference in the recorded benchmark.

| Project element | Portfolio implementation |
| --- | --- |
| Primary model | YOLOv11-n |
| Comparison model | DETR ResNet-50 |
| Detection class | mango |
| Training environment | Google Colab with CUDA |
| Selected checkpoint | best.pt from the tuned YOLO experiment |
| Primary evaluation metrics | Precision, recall, mAP@0.5 and mAP@0.5:0.95 |

## 1. Project Overview

Accurate fruit detection can support fruit-load estimation, harvest planning, labour allocation and orchard management. Manual counting is time-consuming and difficult to scale across large orchards, while orchard imagery presents challenges such as occlusion, clustered fruit, variable scale, dark backgrounds and leaves with similar visual characteristics [1].

The project focused on whether a lightweight one-stage detector could provide strong accuracy and practical processing speed. YOLO treats object detection as a unified prediction problem and is widely used where low-latency detection is important [2]. DETR offers a contrasting Transformer-based approach that predicts objects as a set using learned object queries and bipartite matching [3].

### Project objectives

- Fine-tune YOLOv11-n for mango detection using annotated orchard imagery.
- Compare a baseline YOLO configuration with a tuned experiment.
- Evaluate the selected checkpoint on a held-out test set.
- Compare YOLOv11-n with DETR ResNet-50 using shared accuracy and efficiency measures.
- Document limitations and realistic next steps for deployment-oriented development.

## 2. Dataset and Licensing

The project used a single-class mango object-detection dataset with bounding-box annotations. The YOLO workflow used normalised YOLO text labels, while the DETR workflow used COCO-style JSON annotations [6]. The complete dataset was divided into separate training, validation and test splits so that model selection could be completed before the final held-out test evaluation.

| Dataset split | Images | Purpose |
|---|---|---|
| Training | 1,384 | Model training |
| Validation | 260 | Tuning and model selection |
| Test | 86 | Final held-out evaluation |
| Total | 1,730 | Complete dataset |

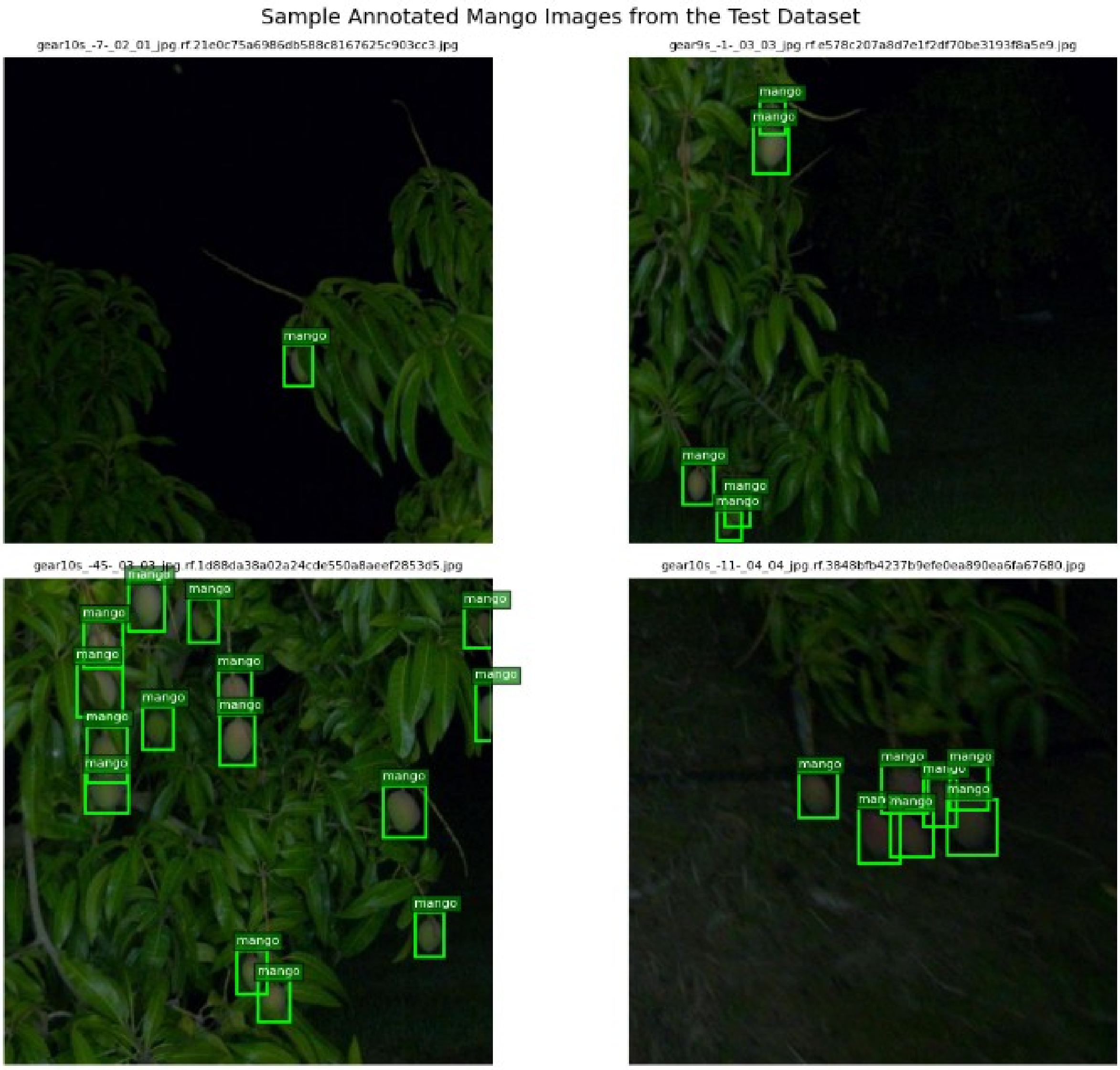



*Figure 1. Ground-truth mango annotations from the held-out test dataset.*

**Dataset attribution**

Dataset source: Roboflow Universe, “mango detection” (weed-mapping/mango-detection-glzls). The source states a Creative Commons Attribution 4.0 International (CC BY 4.0) licence [8]. The full dataset is not redistributed in the public repository; attribution, setup instructions and selected samples are provided instead.

## 3. Methodology

The project workflow consisted of five stages:

1. Prepare the YOLO and COCO dataset formats using the same train, validation and test splits.
2. Fine-tune pretrained YOLOv11-n and DETR ResNet-50 models.
3. Use validation metrics to compare YOLO experiments and select the final YOLO checkpoint.
4. Evaluate the selected models on the held-out test set.
5. Compare quantitative metrics, inference time and qualitative prediction behaviour.

### 3.1 YOLOv11-n configuration

YOLOv11-n was fine-tuned through the Ultralytics implementation [7]. Pretrained weights were used as the starting point. The selected tuned experiment used 640 x 640 input images, batch size 16, the AdamW optimiser and an initial learning rate of 0.0008. Training was planned for 100 epochs and stopped at epoch 77 through early stopping; the best recorded epoch was 57.

| Setting | YOLOv11-n tuned experiment |
|---|---|
| Detector type | CNN-based one-stage detector |
| Input size | 640 x 640 |
| Batch size | 16 |
| Optimiser | AdamW |
| Initial learning rate | 0.0008 |
| Planned epochs | 100 |
| Training stopped | Epoch 77 |
| Best recorded epoch | 57 |
| Selected checkpoint | best.pt |
| Class | mango |

### 3.2 DETR comparison configuration

DETR ResNet-50 was included as a comparison from a different detector family. The model used a ResNet-50 backbone, Transformer encoder-decoder and 100 object queries. It was fine-tuned using COCO-style annotations with batch size 2, a main learning rate of 0.0001 and a backbone learning rate of 0.00001. The recorded evaluation used the preserved epoch 22 checkpoint.

### 3.3 Evaluation measures

**Precision:** The proportion of predicted mango detections that correctly matched ground-truth objects.

**Recall:** The proportion of annotated mangoes detected by the model.

**mAP@0.5:** Mean Average Precision at an Intersection over Union threshold of 0.50.

**mAP@0.5:0.95:** Mean Average Precision across IoU thresholds from 0.50 to 0.95; this is a stricter localisation measure.

**Inference time:** Average prediction-pipeline time per image after warm-up; model loading time was excluded.

# 4. YOLO Experiments and Model Selection

Two YOLOv11-n experiments were compared on the validation split. The baseline used 50 epochs and a learning rate of 0.001. The tuned experiment reduced the learning rate to 0.0008, planned up to 100 epochs and used early stopping. The tuned model achieved the highest recorded mAP@0.5:0.95 and slightly improved precision and recall, so its best.pt checkpoint was selected.

| Experiment | Epochs | LR | mAP50 | mAP50-95 | Precision | Recall |
|---|---|---|---|---|---|---|
| Baseline | 50 | 0.0010 | 0.988831 | 0.701359 | 0.962133 | 0.951333 |
| Tuned (selected) | 100 planned; stop 77 | 0.0008 | 0.990000 | 0.703000 | 0.966000 | 0.956000 |

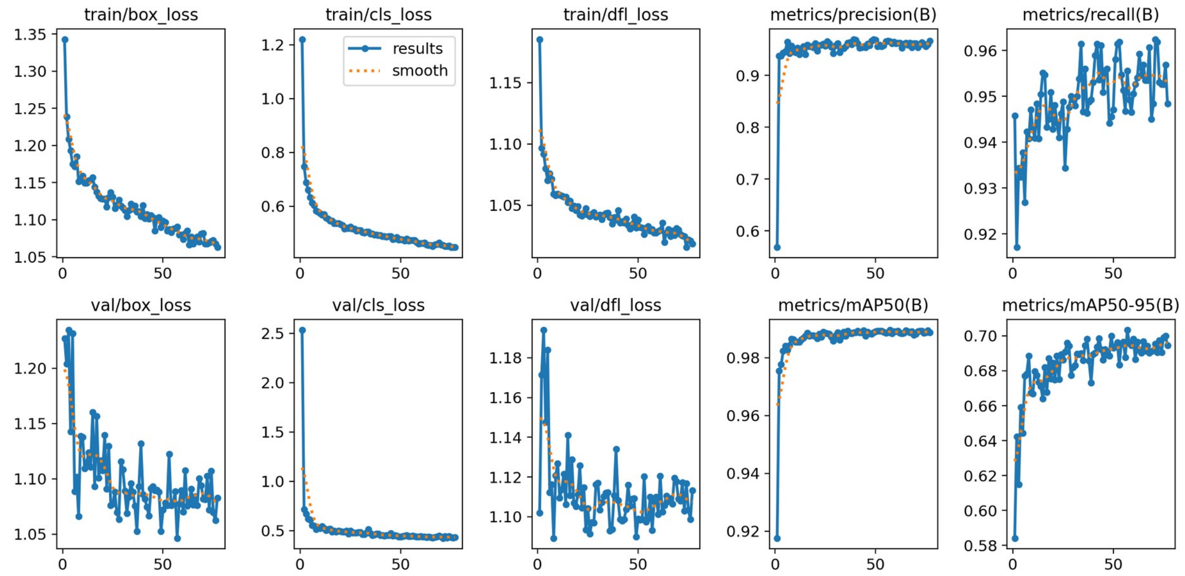


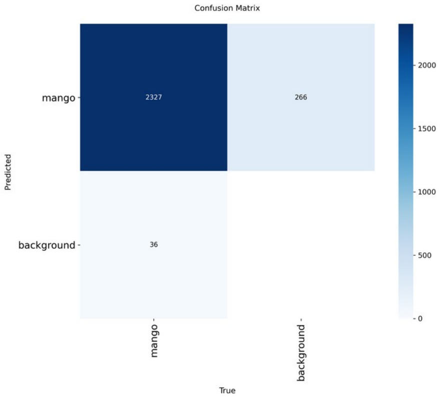

*Figure 2. Training and validation history from the tuned YOLOv11-n experiment.*

# 5. Final YOLOv11-n Evaluation

The selected YOLOv11-n checkpoint was evaluated on the held-out test split containing 86 images and 694 annotated mango instances. The model achieved high recall, indicating that most annotated mangoes were detected, while its precision indicates a comparatively low false-positive rate. The difference between mAP@0.5 and mAP@0.5:0.95 shows that tight localisation remains more demanding than detection at the conventional 0.50 IoU threshold.

| Metric | Held-out test result |
|---|---|
| Precision | 0.936436 |
| Recall | 0.971182 |
| mAP@0.5 | 0.989359 |
| mAP@0.5:0.95 | 0.704345 |
| Test images | 86 |

| Metric | Held-out test result |
|---|---|
| Test instances | 694 |



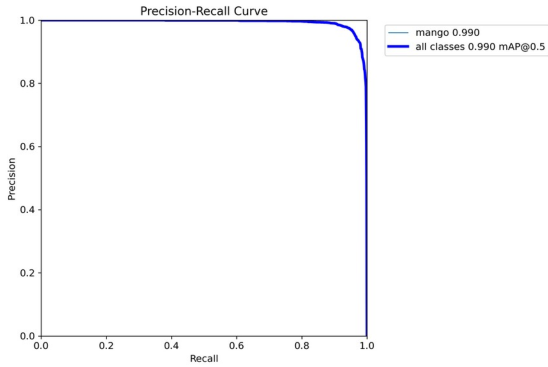



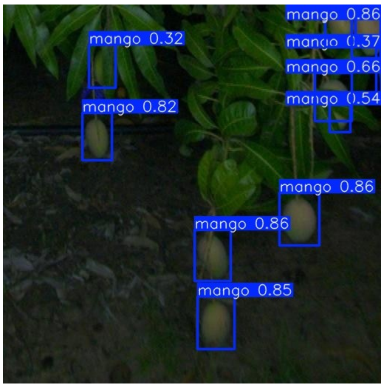

*Figure 3a. YOLOv11-n confusion matrix.*

*Figure 3b. Precision-recall curve.*

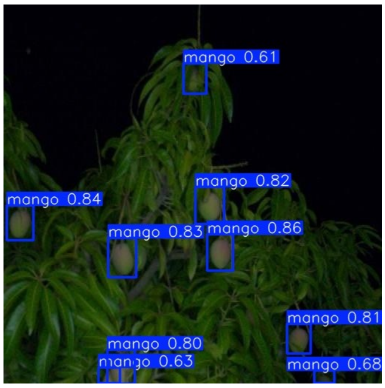
The confusion matrix and precision-recall curve support the numerical test results. The model performed strongly on the single mango class, although the precision-recall curve also illustrates the normal trade-off between retaining detections and filtering lower-confidence predictions.

### Selected YOLO prediction examples


*Prediction example 1*




*Prediction example 2*




*Prediction example 3*

## 6. Comparison with DETR ResNet-50

Both models detected mangoes successfully, but YOLOv11-n led on every shared accuracy metric. The largest difference appeared in precision and mAP@0.5:0.95, indicating that DETR generated more false-positive detections and less precise bounding boxes under stricter localisation thresholds. Recall was similar, showing that both models found most annotated fruit.

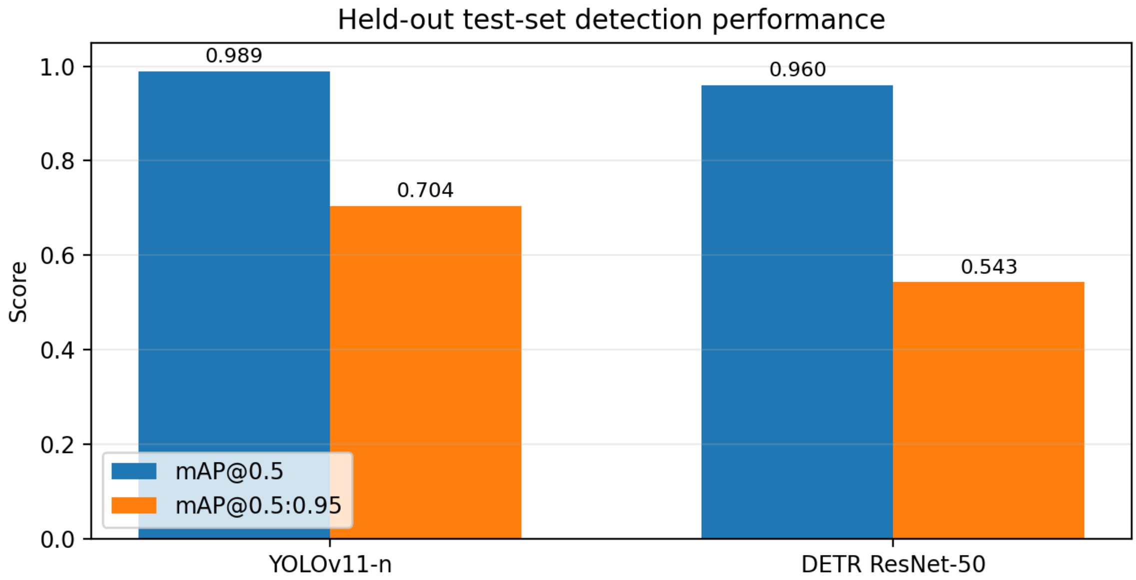



*Figure 4. Shared test-set mAP comparison between YOLOv11-n and DETR ResNet-50.*

| Model | mAP50 | mAP50-95 | Precision | Recall | Avg. ms/image |
|---|---|---|---|---|---|
| YOLOv11-n | 0.989359 | 0.704345 | 0.936436 | 0.971182 | 13.302 ms/image |
| DETR ResNet-50 | 0.960000 | 0.543000 | 0.801909 | 0.968300 | 78.436 ms/image |

In the recorded CUDA benchmark, YOLOv11-n was approximately 5.9 times faster than DETR ResNet-50. This measurement is specific to the tested Colab environment and excludes model loading time; performance can vary on other hardware and software configurations.


*Figure 5a. YOLOv11-n prediction (blue boxes).*

*Figure 5b. DETR ResNet-50 prediction (red boxes).*

The corrected figure labels above reflect the actual model outputs. In the selected examples, YOLOv11-n produced cleaner and more compact localisation, while DETR detected mangoes but showed less precise boxes in some clustered or occluded regions.

## 7. Discussion and Limitations

The results support YOLOv11-n as the stronger model for this dataset. Its high recall indicates that it detected most visible mangoes, while its higher precision and stricter mAP score suggest more dependable localisation than the DETR comparison. The lightweight architecture and lower recorded inference time also make it a more practical starting point for real-time or near-real-time experimentation.

### Important limitations

- The evaluation used one dataset, one mango class and a held-out test set of 86 images.
- The imagery does not represent every orchard, cultivar, season, camera or lighting condition.
- Occluded, clustered, small and poorly illuminated mangoes remain difficult to localise precisely.
- The inference benchmark was recorded in Google Colab and should not be treated as a universal hardware benchmark.
- The model detects fruit but does not assess ripeness, variety, disease, damage or commercial quality.
- A high test score does not by itself establish production readiness or commercial forecasting accuracy.

**Responsible use**

The model is suitable for portfolio demonstration, education and prototype research. It should be validated with broader field data before being used for commercial crop estimation, autonomous harvesting or other consequential agricultural decisions.

## 8. Conclusion and Future Work

YOLOv11-n and DETR ResNet-50 were evaluated for mango detection using the same orchard-image dataset. Both models achieved strong recall, but YOLOv11-n delivered the best overall balance of detection accuracy, localisation quality, precision and inference efficiency. The tuned YOLOv11-n checkpoint was therefore selected as the final portfolio model.

The project demonstrates an end-to-end computer-vision workflow: dataset preparation, transfer learning, experiment comparison, validation-based model selection, held-out testing, visual inspection and transparent discussion of limitations. This makes the project suitable for demonstrating practical skills in Python, deep learning, object detection, experiment tracking and technical communication.

### Recommended next steps

- Collect and test data from additional orchards, cultivars, seasons and lighting conditions.
- Compare YOLOv11-n with YOLOv11-s, YOLOv11-m and larger variants.
- Evaluate longer, stable DETR training using consistent GPU access.
- Add mango counting and fruit-load estimation with duplicate-detection controls.
- Extend the label set to include ripeness, damage or disease categories.
- Deploy the selected model through a FastAPI or Django image-upload application.
- Benchmark latency on local GPUs and edge devices intended for field deployment.

# References


[1] A. Koirala, K. B. Walsh, Z. Wang, and C. McCarthy, “Deep learning for real-time fruit detection and orchard fruit load estimation: Benchmarking of MangoYOLO,” Precision Agriculture, vol. 20, pp. 1107-1135, 2019. doi: 10.1007/s11119-019-09642-0.

[2] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 779-788.

[3] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, “End-to-End Object Detection with Transformers,” in Computer Vision - ECCV 2020, pp. 213-229, 2020.

[4] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks,” IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 39, no. 6, pp. 1137-1149, 2017. doi: 10.1109/TPAMI.2016.2577031.

[5] W. Liu et al., “SSD: Single Shot MultiBox Detector,” in Computer Vision - ECCV 2016, pp. 21-37, 2016.

[6] T.-Y. Lin et al., “Microsoft COCO: Common Objects in Context,” in Computer Vision - ECCV 2014, pp. 740-755, 2014.

[7] Ultralytics, “Ultralytics YOLO Documentation.” https://docs.ultralytics.com/ (accessed May 30, 2026).

[8] Roboflow Universe, “mango detection,” weed-mapping/mango-detection-glzls. https://universe.roboflow.com/weed-mapping/mango-detection-glzls (accessed July 2026). Dataset licence stated by the provider: CC BY 4.0.


**End of report**